

URL Shortener Project Report

1. Introduction

This project is a simple URL Shortener web application developed using the Flask framework. The goal of the project was to understand how a backend web application works in a production-like environment. The application takes a long URL as input and generates a shorter version that redirects to the original URL. This project was completed as part of an internship requirement, where Flask was introduced but database concepts were not deeply covered. Therefore, a self-learning approach was used to integrate database functionality.

2. Problem Understanding

Long URLs are often difficult to share and remember. A URL shortener solves this problem by converting long URLs into short, unique codes. When a user accesses the short URL, they are redirected to the original URL. The application also needed to maintain a history of shortened URLs and track how many times a short URL was used.

3. Initial Approach Using Dictionary

At the beginning of the project, a Python dictionary was used to store URLs. The dictionary mapped long URLs to short codes and short codes back to long URLs. This approach helped in understanding the complete logic of URL shortening without involving database complexity. However, the dictionary-based solution had limitations such as data loss on server restart and lack of persistence. Due to these limitations, switching to a database became necessary.

| 4. Transition to Database and ORM

To solve the persistence problem, a database was introduced. SQLite was chosen because it is lightweight and does not require separate installation. For database interaction, SQLAlchemy ORM was selected. Although I do not have extensive experience with SQLAlchemy, it was chosen because it integrates well with Flask and allows database operations using Python code instead of raw SQL. This made the learning process easier and more aligned with Python programming skills.

| 5. Database Design

A single table was designed to store URL data. The table includes fields for the original URL, the short code, and a click counter. SQLAlchemy models were used to define the table structure. ORM allowed treating database rows as Python objects, which simplified querying and data manipulation.

| 6. Application Workflow

When a user submits a URL, the application first validates the URL format. If the URL already exists in the database, the existing short code is returned. Otherwise, a new unique short code is generated and stored in the database. When a short URL is accessed, the application looks up the original URL and redirects the user while incrementing the click counter. A history page displays all previously shortened URLs along with their short versions and click counts.

| 7. URL Validation

Backend URL validation was implemented to ensure only valid URLs are stored. The validation checks whether the URL contains a proper scheme such as http or https and a valid domain. This prevents invalid or unsafe URLs from being processed by the application.

| 8. Challenges Faced

One major challenge was working with SQLAlchemy without prior experience. Errors related to application context and database table creation were encountered. These issues were resolved by understanding Flask application context and correctly initializing the ORM. This experience improved understanding of backend architecture and debugging techniques.

| 9. Conclusion

This project helped in understanding how Flask applications work with databases in real-world scenarios. Starting with a dictionary-based approach made the logic clear, and transitioning to ORM improved persistence and scalability. Despite limited prior experience with SQLAlchemy, the project successfully demonstrated database integration, URL validation, redirection, and history tracking. Overall, this project enhanced backend development skills and provided practical exposure to production-style Flask applications.